Administrator Manual

Research & Technology Transfer Management System

Version 1.0 | June 2026

Research & Technology Transfer Platform

RTT Platform — Administrator Manual

Research & Technology Transfer Management System Version 1.0 | June 2026

Table of Contents

1. Introduction
2. System Architecture
3. Installation and Deployment
    - 3.1 Server Requirements
    - 3.2 Installation Steps
    - 3.3 Environment Configuration
    - 3.4 Database Setup
    - 3.5 Queue Workers
    - 3.6 Scheduled Tasks (Cron)
    - 3.7 Storage Configuration
    - 3.8 Frontend Build
4. Configuration Reference
    - 4.1 Application Settings
    - 4.2 Email Configuration
    - 4.3 OAuth / SSO Configuration
    - 4.4 Telegram Bot Configuration
    - 4.5 Redis / Cache Configuration
    - 4.6 Session Configuration
5. User and Role Management
    - 5.1 Creating and Managing Users
    - 5.2 Roles and Permissions
    - 5.3 Role Assignment
    - 5.4 Scoped Roles (Category and College)
    - 5.5 Guest User Management
    - 5.6 User Impersonation
6. System Settings

- 6.1 General Settings
- 6.2 Feature Management
- 6.3 Login Notices
- 6.4 Public Site Configuration

7. Proposal Administration
   - 7.1 Proposal Calls Management
   - 7.2 Evaluation Criteria Setup
   - 7.3 Thematic Areas
8. Organizational Structure
   - 8.1 Managing Colleges
   - 8.2 Managing Departments
9. System Monitoring
   - 9.1 System Monitor Dashboard
   - 9.2 Performance Alerts
   - 9.3 Failed Job Management
   - 9.4 Request Metrics
10. Backup and Restore
    - 10.1 Automated Backups
    - 10.2 Manual Backup
    - 10.3 Restoring from Backup
11. Security Administration
    - 11.1 Authentication Security
    - 11.2 Session Security
    - 11.3 Rate Limiting
    - 11.4 Security Headers
    - 11.5 Audit Logging
    - 11.6 Two-Factor Authentication
12. Email Administration
    - 12.1 SMTP Configuration
    - 12.2 Testing Email Delivery
    - 12.3 Broadcast Emails
13. Cache Management
14. Scheduled Commands Reference

15. Maintenance Mode
16. Troubleshooting (Admin)
17. Frequently Asked Questions (Admin)

---

1. Introduction

This Administrator Manual is intended for system administrators and technical staff responsible for deploying, configuring, and maintaining the **RTT Platform** — the Research and Technology Transfer Management System.

This manual covers:

- Server installation and environment configuration
- User and role management
- System settings and feature management
- Security administration
- Backup and restore procedures
- System monitoring and performance management
- Scheduled task management

For end-user instructions, refer to the companion **User Manual**.

Administrator Roles

There are two administrative roles in the system:

Role	Capabilities
Super Admin	Full system access: roles, permissions, features, settings, audit logs, impersonation, all modules
Admin	Settings, public site, login notices, user management, feedback — cannot change roles/permissions or access feature management

> Throughout this manual, actions marked **(Super Admin only)** require the `super_admin` role.

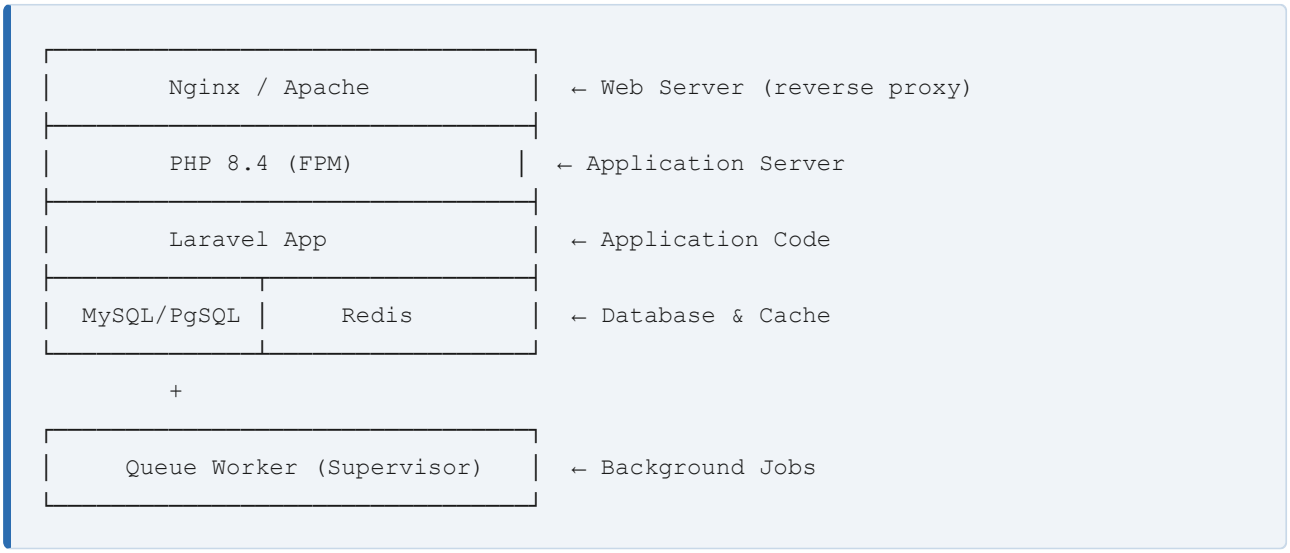
---

2. System Architecture

Technology Stack

Layer	Technology	Version
Backend Framework	Laravel	13.x
PHP	PHP	8.4
Frontend Framework	React	19.x
SPA Bridge	Inertia.js	3.x
CSS Framework	Tailwind CSS	4.x
Authentication	Laravel Sanctum	4.x
Authorization	Spatie Laravel-Permission	Latest
Social Auth	Laravel Socialite	5.x
Testing	PestPHP	4.x
PDF Generation	DomPDF	Latest

Recommended Infrastructure



Key Application Directories

```
/your-app/
├── app/
│   ├── Console/Commands/     ← Scheduled & artisan commands
│   ├── Http/Controllers/     ← Application controllers
│   ├── Http/Middleware/      ← Custom middleware
│   ├── Models/               ← Eloquent models
│   ├── Notifications/        ← Notification classes
│   ├── Policies/             ← Authorization policies
│   └── Services/             ← Service layer
├── config/                   ← Laravel configuration files
├── database/
│   ├── migrations/           ← Database schema migrations
│   └── seeders/              ← Initial data seeders
├── resources/
│   └── js/Pages/             ← React/Inertia page components
├── routes/
│   ├── web.php               ← Web routes
│   ├── api.php               ← API routes
│   └── console.php           ← Console/scheduled routes
├── storage/
│   ├── app/                  ← User-uploaded files
│   ├── logs/                 ← Application logs
│   └── framework/            ← Cache, sessions, compiled views
└── .env                      ← Environment configuration (never commit)
```

---

3. Installation and Deployment

3.1 Server Requirements

Minimum server specifications:

Resource	Minimum	Recommended
CPU	2 cores	4+ cores
RAM	2 GB	4–8 GB
Disk	20 GB	50+ GB (for file uploads and backups)
OS	Ubuntu 22.04 LTS	Ubuntu 22.04 / 24.04 LTS

Required software:

- PHP 8.4 with extensions: `pdo`, `pdo_mysql`, `mbstring`, `xml`, `curl`, `zip`, `gd`, `fileinfo`, `redis`, `intl`
- Composer 2.x
- Node.js 20+ and npm
- MySQL 8.0+ (or PostgreSQL 15+)
- Redis 7+ (for cache, sessions, and queues)
- Nginx or Apache web server
- Supervisor (for queue workers)

Optional but recommended:

- SSL certificate (Let's Encrypt or institutional certificate)
- Elasticsearch (for advanced full-text search)

3.2 Installation Steps

1. Clone the Repository

```
git clone https://github.com/your-org/rtt-platform.git /var/www/rtt-platform
cd /var/www/rtt-platform
```

2. Install PHP Dependencies

```
composer install --no-dev --optimize-autoloader
```

3. Install Node Dependencies and Build Frontend

```
npm install
npm run build
```

4. Copy and Configure Environment File

```
cp .env.example .env
php artisan key:generate
```

Edit `.env` with your configuration (see Section 3.3).

5. Run Database Migrations and Seeders

```
php artisan migrate --force
php artisan db:seed --force
```

6. Set File Permissions

```
chown -R www-data:www-data /var/www/rtt-platform
chmod -R 755 /var/www/rtt-platform/storage
chmod -R 755 /var/www/rtt-platform/bootstrap/cache
```

7. Create Storage Symlink

```
php artisan storage:link
```

8. Configure Nginx

```
server {
    listen 80;
    server_name your-domain.com;
    root /var/www/rtt-platform/public;
    index index.php;

    add_header X-Frame-Options "SAMEORIGIN";
    add_header X-Content-Type-Options "nosniff";

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location = /favicon.ico { access_log off; log_not_found off; }
    location = /robots.txt  { access_log off; log_not_found off; }

    error_page 404 /index.php;

    location ~ \.php$ {
        fastcgi_pass unix:/var/run/php/php8.4-fpm.sock;
        fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;
        include fastcgi_params;
        fastcgi_hide_header X-Powered-By;
    }

    location ~ /\.(?!well-known).* {
        deny all;
    }
}
```

For HTTPS (recommended), add an SSL block and redirect HTTP to HTTPS.

9. Optimize for Production

```
php artisan config:cache
php artisan route:cache
php artisan view:cache
php artisan event:cache
```

3.3 Environment Configuration

The `.env` file controls all environment-specific settings. Below are the critical variables:

```
# Application
APP_NAME="RTT Platform"
APP_ENV=production
APP_KEY=base64:...           # Generated by php artisan key:generate
APP_DEBUG=false              # MUST be false in production
APP_URL=https://your-domain.com

# Database
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=rtt_db
DB_USERNAME=rtt_user
DB_PASSWORD=your_secure_password

# Cache and Sessions
CACHE_DRIVER=redis
SESSION_DRIVER=redis
QUEUE_CONNECTION=redis

# Redis
REDIS_HOST=127.0.0.1
REDIS_PASSWORD=null
REDIS_PORT=6379

# Mail
MAIL_MAILER=smtp
MAIL_HOST=smtp.your-mail-provider.com
MAIL_PORT=587
MAIL_USERNAME=your@email.com
MAIL_PASSWORD=your_mail_password
MAIL_ENCRYPTION=tls
MAIL_FROM_ADDRESS=noreply@your-domain.com
MAIL_FROM_NAME="RTT Platform"

# Storage (local or S3)
FILESYSTEM_DISK=local
# For AWS S3:
# FILESYSTEM_DISK=s3
# AWS_ACCESS_KEY_ID=...
# AWS_SECRET_ACCESS_KEY=...
# AWS_DEFAULT_REGION=...
# AWS_BUCKET=...

# Microsoft OAuth (if using SSO)
AZURE_CLIENT_ID=your-client-id
AZURE_CLIENT_SECRET=your-client-secret
AZURE_TENANT_ID=your-tenant-id
AZURE_REDIRECT_URI=https://your-domain.com/auth/callback/azure
```

```
# Telegram Bot (if using Telegram notifications)
TELEGRAM_BOT_TOKEN=your-bot-token
TELEGRAM_WEBHOOK_URL=https://your-domain.com/telegram/webhook
```

> **Security:** Never commit `.env` to version control. Ensure it is owned by the web server user and not publicly accessible.

3.4 Database Setup

Create the Database

```
CREATE DATABASE rtt_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'rtt_user'@'localhost' IDENTIFIED BY 'your_secure_password';
GRANT ALL PRIVILEGES ON rtt_db.* TO 'rtt_user'@'localhost';
FLUSH PRIVILEGES;
```

Run Migrations

```
php artisan migrate --force
```

Seed Initial Data

```
php artisan db:seed --force
```

The seeder creates:

- Default roles and permissions
- Default system settings
- Sample colleges and departments (if included in seeder)
- Initial super admin account (if configured in seeder)

Creating the First Super Admin

If no admin was created by the seeder, create one via Artisan:

```
php artisan tinker --execute '
$user = \App\Models\User::create([
    "name" => "System Administrator",
    "email" => "admin@your-domain.com",
    "password" => bcrypt("ChangeThisPassword!"),
    "status" => "active",
]);
$user->assignRole("super_admin");
echo "Admin created: " . $user->email;
'
```

> **Important:** Change the password immediately after first login.

3.5 Queue Workers

Background jobs (notifications, backups, report generation) require a running queue worker.

Configure Supervisor

Create `/etc/supervisor/conf.d/rtt-worker.conf`:

```
[program:rtt-worker]
process_name=%(program_name)s_%(process_num)02d
command=php /var/www/rtt-platform/artisan queue:work redis --sleep=3 --tries=3 --max-ti
autostart=true
autorestart=true
stopasgroup=true
killasgroup=true
user=www-data
numprocs=2
redirect_stderr=true
stdout_logfile=/var/www/rtt-platform/storage/logs/worker.log
stopwaitsecs=3600
```

Start Supervisor

```
sudo supervisorctl reread
sudo supervisorctl update
sudo supervisorctl start rtt-worker:*
```

Monitor Queue Workers

```
sudo supervisorctl status
php artisan queue:monitor redis
```

3.6 Scheduled Tasks (Cron)

Add the Laravel scheduler to the server cron:

```
crontab -e -u www-data
```

Add this line:

```
* * * * * cd /var/www/rtt-platform && php artisan schedule:run >> /dev/null 2>&1
```

This runs the Laravel scheduler every minute, which then executes scheduled commands at their configured intervals.

Scheduled commands and their schedules:

Command	Schedule	Purpose
`rtt:deadline-reminders`	Daily at 08:00	Send 7/3/1-day reminders to PIs
`rtt:expire-calls`	Hourly	Mark expired proposal calls
`users:deactivate-expired-guests`	Daily	Deactivate expired guest accounts
`backup:run`	Daily at configured time	Automated database backup
`rtt:collect-metrics`	Every 5 minutes	Collect system performance metrics
`rtt:cleanup-telegram-tokens`	Daily	Remove expired Telegram tokens
`rtt:warm-dashboard-cache`	Every 30 minutes	Pre-compute dashboard statistics

3.7 Storage Configuration

Local Storage

By default, files are stored in `storage/app/`. Ensure this directory is writable by the web server and run:

```
php artisan storage:link
```

This creates a symlink from `public/storage` → `storage/app/public` for publicly accessible files.

AWS S3 Storage (Recommended for Production)

For production deployments with multiple servers or high file volume, use S3:

1. Create an S3 bucket in your AWS region.
2. Create an IAM user with S3 read/write permissions to that bucket.
3. Configure `.env`:

```
FILESYSTEM_DISK=s3
AWS_ACCESS_KEY_ID=AKIA...
AWS_SECRET_ACCESS_KEY=...
AWS_DEFAULT_REGION=us-east-1
AWS_BUCKET=rtt-platform-files
```

Private File Access

Most uploaded files (proposals, reports, CVs) are stored in the **private** area and served through authenticated routes with time-limited signed URLs. Do not expose the `storage/app/private` directory publicly.

3.8 Frontend Build

For production:

```
npm install
npm run build
```

This compiles React/Inertia components and Tailwind CSS into `public/build/`.

For development (with hot reload):

```
composer run dev
# or
npm run dev
```

> **Important:** Always run `npm run build` after pulling new code in production. Users seeing a "Vite manifest not found" error indicates the build needs to be regenerated.

---

4. Configuration Reference

4.1 Application Settings

Most runtime settings are configurable through the admin UI at **Settings** → **Settings** without requiring code changes. See Section 6.1 for full details.

Settings stored in the `settings` database table include:

- System name, tagline, logo
- Sidebar color
- Announcement banner
- Footer text
- Session timeout
- Self-registration toggle
- Password login toggle
- Auto-backup settings
- Maintenance mode

4.2 Email Configuration

Configure via `.env` as shown in Section 3.3, or update through the admin UI at **Settings** → **Email Configuration**.

Supported mail drivers:

- `smtp` — Standard SMTP (most common)
- `mailgun` — Mailgun API
- `ses` — Amazon SES
- `postmark` — Postmark API
- `log` — Log to file (development only)
- `array` — No-op (testing only)

> **Critical:** Use the `log` driver only in development. In production, always configure a real SMTP or transactional email service.

4.3 OAuth / SSO Configuration

Microsoft Azure AD

1. Register an application in the Azure Portal.
2. Under **Authentication**, add `https://your-domain.com/auth/callback/azure` as a Redirect URI.
3. Under **Certificates & Secrets**, create a client secret.
4. Under **API Permissions**, add `User.Read` (Microsoft Graph).
5. Configure `.env`:

```
AZURE_CLIENT_ID=your-application-client-id
AZURE_CLIENT_SECRET=your-client-secret-value
AZURE_TENANT_ID=your-directory-tenant-id
AZURE_REDIRECT_URI=https://your-domain.com/auth/callback/azure
```

6. In **Settings** → **Settings**, ensure **Microsoft SSO** is enabled.

Testing SSO

Use a test Azure AD account. Check `storage/logs/laravel.log` for OAuth errors if login fails.

4.4 Telegram Bot Configuration

1. Open Telegram and search for `@BotFather`.
2. Send `/newbot` and follow instructions to create a bot.
3. Copy the **Bot Token** provided by BotFather.
4. Configure `.env`:

```
TELEGRAM_BOT_TOKEN=1234567890:ABCdefGHIjklMNOpqrSTUVwxyz
TELEGRAM_WEBHOOK_URL=https://your-domain.com/telegram/webhook
```

5. Register the webhook:

```
php artisan telegram:set-webhook
# or via HTTPS:
# curl "https://api.telegram.org/bot{TOKEN}/setWebhook?url=https://your-domain.com/t
```

> **Requirement:** The webhook URL must be HTTPS. Local development requires a tunnel (e.g., ngrok).

4.5 Redis / Cache Configuration

Redis is required for:

- Session storage
- Application cache
- Queue management
- Rate limiting counters

Redis connection in `.env`:

```
REDIS_HOST=127.0.0.1
REDIS_PASSWORD=null
REDIS_PORT=6379
REDIS_CLIENT=predis
```

For Redis with password:

```
REDIS_PASSWORD=your-redis-password
```

For Redis Sentinel or Cluster, configure `config/database.php` directly.

4.6 Session Configuration

Session settings in `.env`:

```
SESSION_DRIVER=redis
SESSION_LIFETIME=120          # Minutes of inactivity before logout
SESSION_ENCRYPT=true          # Encrypt session data (recommended)
SESSION_SECURE_COOKIE=true    # HTTPS-only cookies (required for production)
SESSION_SAME_SITE=lax        # CSRF protection
```

> The `SESSION_LIFETIME` here is the absolute session timeout. The in-app warning dialog timeout is configured separately in **Settings → Settings → Session Timeout**.

5. User and Role Management

5.1 Creating and Managing Users

Path: Administration → Users

See the User Manual Section 19 for the full user management interface. As an administrator, you have the following additional capabilities:

Importing Users in Bulk

1. Go to **Administration → Users → Import**.
2. Download the **CSV Template**.
3. Fill in the following columns:
   - `name` (required)
   - `email` (required, must be unique)
   - `employee_id` (optional)
   - `college` (college name or code)
   - `department` (department name or code)
   - `role` (role name, e.g., `staff`, `research_officer`)
   - `title`
   - `phone`
   - `gender` (male/female/other)
   - `status` (active/inactive/guest)
4. Upload and review the import preview.
5. Confirm the import.

Import notes:

- Emails must be unique; duplicate emails are skipped with a warning.
- Invalid roles are ignored; the user is created without a role.
- A welcome email is sent to each imported user.

Exporting Users

Go to **Administration → Users** and click **Export** to download the current filtered list as CSV.

5.2 Roles and Permissions

Path (Super Admin only): Administration → Roles / Administration → Permissions

The RTT Platform uses **Spatie Laravel-Permission** for role-based access control.

Built-in Roles

Role	Slug	Description
Super Admin	`super_admin`	Unrestricted access to all features
Admin	`admin`	Settings and user management
Central Officer	`central_officer`	System-wide proposal coordination
Research Officer	`research_officer`	Research category proposals
Community Officer	`community_officer`	Community engagement proposals
Technology Officer	`technology_officer`	Technology transfer proposals
College Officer	`college_officer`	College-scoped management
Finance Officer	`finance_officer`	All budget-related features
Support Officer	`support_officer`	General support
Staff	`staff`	Proposal submission
Guest	`guest`	Read-only public access

Viewing Permissions

1. Go to **Administration** → **Permissions**.
2. All system permissions are listed by resource and action.

Permission naming convention: `resource.action`

Examples:

- `proposals.create` — can create proposals
- `proposals.view_any` — can see all proposals
- `proposals.validate` — can validate proposals
- `users.impersonate` — can impersonate other users
- `reports.view_any` — can access the reports module

Creating Custom Roles (Super Admin only)

1. Go to **Administration** → **Roles**.
2. Click **New Role**.
3. Enter a **Role Name** (slug — no spaces, lowercase, underscores allowed).
4. Select the **Permissions** to assign to this role.
5. Click **Save**.

> **Warning:** Be careful when modifying or removing permissions from existing roles. Users with active sessions may need to log out and back in for changes to take effect.

Editing Role Permissions

1. Go to **Administration** → **Roles**.
2. Click **Edit** next to a role.
3. Adjust the selected permissions.
4. Click **Save**.

> **Note:** Changes to the `super_admin` role are not recommended and may be restricted.

5.3 Role Assignment

To assign roles to a user:

1. Go to **Administration** → **Users**.
2. Click the user's name.
3. In the **Roles** section, click **Edit Roles**.
4. Select or deselect roles.
5. Click **Save**.

Multiple roles: A user can have multiple roles. Their effective permissions are the union of all assigned roles' permissions.

5.4 Scoped Roles (Category and College)

Some roles automatically scope data access:

Category-Scoped Roles

Users with `research_officer`, `community_officer`, or `technology_officer` roles only see proposals in their assigned category. The category is determined by their role name.

- `research_officer` → sees only **Research** proposals
- `community_officer` → sees only **Community Engagement** proposals
- `technology_officer` → sees only **Technology Transfer** proposals

This is enforced by the `EnsureCategoryScope` middleware.

College-Scoped Roles

Users with the `college_officer` role only see data from their assigned college. The college is determined by the user's **College** profile field.

To set a user's college:

1. Edit the user.
2. Set the **College** field.
3. Save.

This is enforced by the `EnsureCollegeScope` middleware.

5.5 Guest User Management

Guest users have limited, read-only access for a defined period.

Guest account features:

- Status: `guest`
- Automatic deactivation when their expiry date passes (via the `users:deactivate-expired-guests` scheduled command)
- Access limited to public-facing pages and data

To create a guest account:

1. Create a new user.
2. Set **Status** to `Guest`.
3. Set an **Expiry Date** for when access should end.
4. Assign the `guest` role.
5. Save.

5.6 User Impersonation

(Super Admin only)

Impersonation allows a Super Admin to log in as another user to diagnose issues or provide support.

To impersonate:

1. Go to **Administration** → **Users**.
2. Find the target user.
3. Click the **Impersonate** button (eye icon).
4. You are now browsing as that user.

To end impersonation:

- Click **Stop Impersonating** in the top notification bar.

Audit: All impersonation sessions are logged in the Audit Log with the impersonating admin's identity and all actions taken.

---

6. System Settings

6.1 General Settings

Path: Settings → Settings

These settings control the overall appearance and behavior of the platform.

Setting	Key	Description
System Name	`system_name`	Displayed in header, emails, and browser tab
Tagline	`tagline`	Shown on login page below the system name
Logo File	`logo`	Uploaded image (PNG/SVG)
Logo Width	`logo_width`	Pixel width of logo
Logo Height	`logo_height`	Pixel height of logo
Sidebar Color	`sidebar_color`	Hex color for the navigation sidebar
Announcement	`announcement`	Banner message shown to all logged-in users
Announcement Active	`announcement_active`	Show/hide the banner
Footer Text	`footer_text`	Text in the page footer
Feedback Popup	`feedback_enabled`	Show/hide feedback widget
Session Timeout	`session_timeout`	Minutes of inactivity before warning dialog
Self-Registration	`allow_self_registration`	Allow new users to sign up
Password Login	`allow_password_login`	Enable email/password authentication
Auto-Backup	`auto_backup`	Enable scheduled automated backups
Backup Time	`backup_time`	Time (HH:MM) to run daily backup
Maintenance Mode	`maintenance_mode`	Take site offline for maintenance
Max Concurrent Sessions	`max_sessions`	Maximum simultaneous logins per user (0 = unlimited)

After changing settings: Changes take effect immediately after saving. Some changes (logo, sidebar color) may require a browser refresh.

6.2 Feature Management

(Super Admin only)

Path: Administration → Feature Management

Enable or disable major system modules. Each feature has:

- A **master toggle** (enable/disable the entire module)
- **Sub-feature toggles** for granular control within the module

[Screenshot: Feature Management Page]

Available Features

Feature Key	Controls
`finance`	Budget allocations, requests, disbursements, settlements
`monitoring`	Midterm evaluations, completion workshops, research outputs
`reports`	Report generation, analytics dashboard
`complaints`	Complaint submission and resolution
`similarity`	Similarity/plagiarism detection
`external_funds`	External funding opportunities and applications
`conferences`	Conference management and abstract submissions
`repository`	Public research output repository
`self_registration`	Public user self-registration

Effect of disabling a feature:

- The feature's menu items are hidden from all users.
- Routes for the feature return a 403 error if accessed directly.
- Data is preserved — re-enabling the feature restores full access.

To toggle a feature:

1. Click the toggle switch.
2. Confirm the dialog prompt.
3. Change takes effect immediately for all users.

6.3 Login Notices

Path: Settings → Login Notices

Login notices appear on the login page. Use them for:

- System maintenance announcements

- Policy updates
- Upcoming deadlines
- System outage warnings

To create a notice:

1. Click **New Notice**.
2. Enter the message (supports basic HTML if needed).
3. Set a **Publish At** date/time (or leave blank for immediate).
4. Toggle **Active** to make it visible.
5. Click **Save**.

Multiple active notices stack on the login page in order of creation.

6.4 Public Site Configuration

Path: Settings → Public Site

Configure content displayed on the public-facing pages (landing page, about section, contact details, etc.). Changes here affect what unauthenticated visitors see without requiring code changes.

---

7. Proposal Administration

7.1 Proposal Calls Management

Path: Proposals → Proposal Calls (admin view)

Proposal calls are the mechanism by which researchers are invited to submit proposals.

Creating a Proposal Call

1. Go to **Proposals** → **Proposal Calls**.
2. Click **New Proposal Call**.
3. Fill in:
   - **Title** — descriptive name for the call
   - **Slug** — URL-friendly identifier (auto-generated from title)
   - **Academic Year** — e.g., "2025-2026"
   - **Category** — Research / Community / Technology / Innovation
   - **Eligibility Criteria** — who can apply (rich text)

- **Guidelines** — upload a PDF guidelines document
- **Start Date** — when submissions open
- **End Date** — submission deadline
- **Max Proposals per PI** — limit per researcher (0 = unlimited)
- **Status** — Draft / Active / Closed
- **Is Open** — toggle to accept submissions

4. Click **Save**.

Managing Call Statuses

Status	Meaning
Draft	Not visible to researchers; still being set up
Active	Visible and open for submissions
Expired	End date has passed (automatically set by scheduler)
Closed	Manually closed by an officer

To activate a call:

1. Open the call in edit mode.
2. Set Status to **Active** and toggle **Is Open** to on.
3. Save.

To close a call early:

1. Open the call.
2. Toggle **Is Open** to off, or set Status to **Closed**.
3. Save.

Expiring Calls

The `rtt:expire-calls` command runs hourly and automatically sets calls to **Expired** when their `end_date` passes. You can also run this manually:

```
php artisan rtt:expire-calls
```

7.2 Evaluation Criteria Setup

Path: Review & Evaluation → Evaluation Criteria

Evaluation criteria define the questions and scoring rubrics used during blind review.

Structure:

```
Evaluation Type
  └── Category (e.g., Scientific Merit)
        └── Question (e.g., "Is the methodology sound?", score 0-10)
```

Creating an Evaluation Type

1. Click **New Evaluation Type**.
2. Enter a **Name** (e.g., "Standard Research Evaluation").
3. Set it as the **Default** if it should be used for all blind reviews.
4. Click **Save**.

Adding Categories to an Evaluation Type

1. Open an evaluation type.
2. Click **Add Category**.
3. Enter the **Category Name** and an optional description.
4. Set the **Weight** (relative importance for scoring, optional).
5. Click **Save**.

Adding Questions to a Category

1. Open a category.
2. Click **Add Question**.
3. Enter the **Question Text**.
4. Set the **Maximum Score**.
5. Add optional **Scoring Guidance** (descriptor for each score level).
6. Click **Save**.

> **Tip:** Create evaluation types specific to each proposal category (Research, Community, Technology) if the review criteria differ significantly.

7.3 Thematic Areas

(Super Admin only)

Path: Administration → Thematic Areas

Thematic areas classify research under institutional priority topics. They appear as optional metadata on proposals and in the public repository.

To add a thematic area:

1. Click **New Thematic Area**.
2. Enter the **Name** and optional **Description**.
3. Toggle **Active** to make it available for selection.
4. Click **Save**.

---

8. Organizational Structure

8.1 Managing Colleges

Path: Administration → Colleges

Colleges represent the top-level academic units in your institution.

To add a college:

1. Click **New College**.
2. Enter:
   - **Name** — full official name
   - **Code** — short abbreviation (e.g., "CENG", "CHSS")
3. Click **Save**.

> Colleges cannot be deleted if they have associated users, departments, or proposals. Deactivate them instead.

8.2 Managing Departments

Path: Administration → Departments

Departments belong to colleges.

To add a department:

1. Click **New Department**.
2. Enter:
   - **Name** — department full name
   - **Code** — short abbreviation

- **College** — select the parent college

3. Click **Save**.

---

9. System Monitoring

9.1 System Monitor Dashboard

Path: Administration → System Monitor

Who can access: Central Officer, Admin, Super Admin

[Screenshot: System Monitor Dashboard]

The System Monitor provides real-time visibility into server and application health.

Metrics collected (every 5 minutes):

- **CPU Usage %** — current server CPU utilization
- **Memory Usage %** — RAM used vs. available
- **Disk Usage %** — storage used vs. available
- **Requests per Minute** — API/page request rate
- **Average Response Time** — mean time to serve requests
- **Queue Depth** — number of pending background jobs
- **Failed Jobs** — count of jobs that have failed

Historical view: Metrics are stored over time and displayed as trend charts. Select time windows (last hour, 24 hours, 7 days) to see trends.

9.2 Performance Alerts

The system automatically generates alerts when metrics exceed thresholds:

Metric	Default Alert Threshold
CPU Usage	> 85%
Memory Usage	> 90%
Disk Usage	> 80%
Failed Jobs	> 10 in queue
Response Time	> 2000ms average

Alert statuses:

- **Active** — threshold exceeded; needs attention
- **Acknowledged** — admin has seen and noted the alert
- **Resolved** — metric has returned to normal

To acknowledge an alert:

1. Click **Acknowledge** on the alert.
2. Add a note explaining the action taken.
3. Save.

Alert notifications: Alerts are sent to admin users who have **System Alerts** enabled in their notification preferences.

To configure alert preferences:

1. Go to your Profile → Notification Settings.
2. Toggle **System Alerts** on/off.

9.3 Failed Job Management

Background jobs that fail after all retry attempts are moved to the **failed_jobs** table and shown in the System Monitor.

Common causes of failed jobs:

- Mail server unreachable (email notifications)
- S3/storage unavailable (file operations)
- Database timeout (during heavy report generation)
- External API timeout (Telegram notifications)

To retry a failed job:

1. Go to **System Monitor** → **Failed Jobs**.
2. Click **Retry** on the failed job.

To retry all failed jobs:

```
php artisan queue:retry all
```

To view failed job details: Click on the job to see:

- Exception message
- Stack trace
- Job payload (data that was being processed)

- Failed at timestamp

To clear all failed jobs (use with caution):

```
php artisan queue:flush
```

9.4 Request Metrics

The `TrackRequestMetrics` middleware logs performance data for each HTTP request. This data powers the request statistics section of the System Monitor.

Data captured per request:

- URL and HTTP method
- Response time (milliseconds)
- HTTP status code
- User ID (if authenticated)
- Timestamp

This data is used to identify slow endpoints and usage patterns.

---

10. Backup and Restore

10.1 Automated Backups

Path: Settings → Settings → Auto-Backup

When enabled, the system runs `backup:run` daily at the configured time.

What is backed up:

- Full MySQL/PostgreSQL database dump
- Uploaded files in `storage/app/` (if configured)

Backup storage:

- By default, backups are stored in `storage/app/backups/`
- Configure S3 or another remote storage for production backups

To enable automated backups:

1. Go to **Settings** → **Settings**.
2. Toggle **Auto-Backup** to on.
3. Set the **Backup Time** (e.g., 02:00 for 2:00 AM).

4. Click **Save Settings**.

10.2 Manual Backup

To trigger an immediate backup:

```
php artisan backup:run
```

To back up database only (no files):

```
php artisan backup:run --only-db
```

To back up files only (no database):

```
php artisan backup:run --only-files
```

To list available backups:

```
php artisan backup:list
```

10.3 Restoring from Backup

> **Warning:** Restoring from a backup overwrites the current database. This is a destructive operation. Always test on a staging environment first.

Database Restore (MySQL)

1. Locate the backup file (`.sql.gz` or `.sql`).
2. If gzipped, extract it:

   ```
   gunzip backup_file.sql.gz
   ```

3. Restore the database:

   ```
   mysql -u rtt_user -p rtt_db < backup_file.sql
   ```

File Restore

1. Locate the file backup archive.
2. Extract to the storage directory:

```
tar -xzf files_backup.tar.gz -C /var/www/rtt-platform/storage/app/
```

3. Fix permissions:

```
chown -R www-data:www-data /var/www/rtt-platform/storage/
```

Post-Restore Steps

After restoring:

```
php artisan cache:clear
php artisan config:cache
php artisan route:cache
php artisan view:cache
```

---

11. Security Administration

11.1 Authentication Security

Disable email/password login (force SSO):

1. Go to **Settings** → **Settings**.
2. Toggle **Allow Password Login** to off.
3. Save.

When password login is disabled, users must use Microsoft SSO or Telegram to log in.

Self-registration control:

1. Go to **Settings** → **Settings**.
2. Toggle **Allow Self-Registration** to on or off.
3. Save.

When self-registration is off, only administrators can create user accounts.

11.2 Session Security

Session timeout: Configure in **Settings** → **Settings** → **Session Timeout**. This controls the in-app inactivity warning dialog (separate from the server-side session lifetime in `.env`).

Max concurrent sessions: Set in **Settings** → **Settings** → **Max Concurrent Sessions**. When a user reaches the limit, their oldest session is automatically invalidated.

Recommended `.env` security settings for production:

```
SESSION_SECURE_COOKIE=true    # HTTPS only
SESSION_ENCRYPT=true          # Encrypt session data
SESSION_SAME_SITE=lax        # CSRF protection
APP_DEBUG=false               # Never true in production
```

11.3 Rate Limiting

Rate limits are enforced by middleware and protect against brute-force attacks and abuse.

Endpoint	Limit	Window
Login	30 attempts	Per minute per IP
2FA verification	10 attempts	Per minute per IP
Password reset	30 requests	Per hour per IP
Email verification	30 requests	Per hour per IP
File uploads	60 requests	Per minute per user
Authenticated API	300 requests	Per minute per user
Telegram webhook	30 requests	Per minute per IP

These limits are defined in `routes/api.php` and `routes/web.php` using Laravel's `throttle` middleware. To adjust limits, edit the throttle parameters in those files and clear the route cache:

```
php artisan route:cache
```

11.4 Security Headers

The `SecurityHeaders` middleware adds the following HTTP security headers to all responses:

- `X-Frame-Options: SAMEORIGIN` — prevents clickjacking
- `X-Content-Type-Options: nosniff` — prevents MIME sniffing
- `X-XSS-Protection: 1; mode=block` — basic XSS protection
- `Referrer-Policy: strict-origin-when-cross-origin`
- `Content-Security-Policy` — restricts resource loading

Review and customize headers in `app/Http/Middleware/SecurityHeaders.php` for your institution's requirements.

11.5 Audit Logging

All significant actions are logged to the `audit_logs` table. Logs are:

- **Immutable** — the `LogImmutableException` is thrown on any attempted update or delete
- **Comprehensive** — captures actor, IP address, action type, affected record, old/new values
- **Searchable** — filterable by user, date, action type, and record type

To view audit logs:

1. Go to **Administration** → **Audit Logs**.
2. Use filters to narrow results.

To export audit logs: Click **Export** to download filtered logs as CSV.

Log retention: Logs are retained indefinitely by default. For compliance, implement a retention policy via a scheduled command that archives old logs to external storage.

11.6 Two-Factor Authentication

2FA is optional and per-user. Users can enable it in their profile.

How it works:

1. User enables 2FA in profile settings.
2. On next login, after credentials are verified, a 6-digit code is sent to the user's email.
3. User enters the code to complete login.
4. Rate limit: 10 attempts per minute per IP.

To force 2FA for all users (requires code change): Modify the `AuthService` or `LoginController` to enforce 2FA validation for all authenticated sessions. Contact your developer.

---

12. Email Administration

12.1 SMTP Configuration

Configure via `.env` as described in Section 3.3. Runtime SMTP configuration changes can also be made through the admin UI.

Key `.env` values:

```
MAIL_MAILER=smtp
MAIL_HOST=smtp.your-provider.com
MAIL_PORT=587
MAIL_USERNAME=your-username
MAIL_PASSWORD=your-password
MAIL_ENCRYPTION=tls
MAIL_FROM_ADDRESS=noreply@your-domain.com
MAIL_FROM_NAME="RTT Platform"
```

Common SMTP providers and settings:

Provider	Host	Port	Encryption
Gmail	smtp.gmail.com	587	TLS
Outlook/365	smtp.office365.com	587	TLS
SendGrid	smtp.sendgrid.net	587	TLS
Mailgun	smtp.mailgun.org	587	TLS
Amazon SES	email-smtp.us-east-1.amazonaws.com	587	TLS

> **Gmail note:** Requires an App Password (not your regular password) if 2-Step Verification is enabled on the Google account.

12.2 Testing Email Delivery

After configuring SMTP, send a test email:

1. Go to **Settings** → **Email Configuration**.
2. Enter a **Test Recipient** email address.
3. Click **Send Test Email**.
4. Check the inbox and `storage/logs/laravel.log` if the email is not received.

Via Artisan:

```
php artisan tinker --execute '
Mail::raw("This is a test email from RTT Platform.", function($message) {
    $message->to("your@email.com")->subject("RTT Test Email");
});
echo "Email sent.";
'
```

12.3 Broadcast Emails

Administrators can send mass emails to users.

> **Important:** Broadcast emails are **not queued** by design (to ensure immediate delivery). Sending to a large user list may take significant time. Plan accordingly.

To send a broadcast email:

1. Go to **Administration** → **Users**.
2. Select the recipient group (all users, specific roles, specific college).
3. Click **Broadcast Email**.
4. Compose subject and body.
5. Confirm and send.

> **Note:** The user's email notification preferences (if any) do not suppress broadcast emails — they go to all selected recipients.

---

13. Cache Management

The platform uses Redis for caching. Caches automatically expire and refresh, but manual clearing is sometimes needed after configuration changes or deployments.

Cache keys used:

- Application settings
- Permission/role data (Spatie)
- Dashboard widget counts
- Report snapshots
- System metrics aggregates

To clear all application caches:

Via admin UI:

1. Go to **Administration** → **System Monitor**.
2. Click **Clear Cache**.

Via Artisan:

```
php artisan cache:clear
php artisan config:clear
php artisan route:clear
php artisan view:clear
```

To rebuild the cache after clearing:

```
php artisan config:cache
php artisan route:cache
php artisan view:cache
php artisan rtt:warm-dashboard-cache
```

Permission cache: Spatie's permission cache must be cleared after any role/permission change:

```
php artisan permission:cache-reset
```

---

14. Scheduled Commands Reference

All scheduled commands can be run manually as needed:

Artisan Command	Description	Run Manually When
`php artisan rtt:deadline-reminders`	Send proposal deadline reminders	When testing notification delivery
`php artisan rtt:expire-calls`	Expire past-deadline proposal calls	After a call's end date passes and it hasn't expired
`php artisan users:deactivate-expired-guests`	Deactivate expired guest accounts	After guest accounts should have expired
`php artisan backup:run`	Run a full backup	Before major system changes
`php artisan backup:list`	List all available backups	Audit / verification
`php artisan rtt:warm-dashboard-cache`	Pre-compute dashboard stats	After heavy data imports
`php artisan queue:work`	Start processing queue jobs	When queue worker is down
`php artisan queue:retry all`	Retry all failed jobs	After resolving the root cause of failures
`php artisan queue:flush`	Delete all failed jobs	Housekeeping
`php artisan permission:cache-reset`	Clear Spatie permission cache	After role/permission changes
`php artisan schedule:list`	Show all scheduled commands and their schedules	Audit scheduled tasks
`php artisan schedule:run`	Run due scheduled commands now	Testing the scheduler

---

15. Maintenance Mode

Via admin UI:

1. Go to **Settings** → **Settings**.
2. Toggle **Maintenance Mode** to on.
3. Click **Save Settings**.

While in maintenance mode:

- All non-admin users see a maintenance page.
- Admins and super admins can still access the system.

Via Artisan (for server-side maintenance):

```
# Take down for all users including admins
php artisan down --message="Scheduled maintenance until 3:00 PM."

# Allow specific IPs through (e.g., admin IP)
php artisan down --allow=203.0.113.1

# Bring back up
php artisan up
```

> **Note:** `php artisan down` bypasses the in-app maintenance mode and takes the entire Laravel application offline. Use `php artisan up` to restore it.

---

16. Troubleshooting (Admin)

Application Returns 500 Error

1. Check `storage/logs/laravel.log` for the exception.
2. Ensure `APP_DEBUG=false` in production (errors should not be shown to users).
3. Check file permissions: `storage/` and `bootstrap/cache/` must be writable.
4. Run `php artisan config:clear && php artisan cache:clear`.

Queue Jobs Not Processing

1. Check Supervisor status: `sudo supervisorctl status`
2. If workers are stopped: `sudo supervisorctl start rtt-worker:*`
3. Check worker logs: `tail -f storage/logs/worker.log`
4. Verify Redis is running: `redis-cli ping` (should return `PONG`)

Scheduled Tasks Not Running

1. Verify cron is active: `crontab -l -u www-data`
2. Test the scheduler: `php artisan schedule:run --verbose`
3. Check system time matches expected timezone (set `APP_TIMEZONE` in `.env`)

4. Review `storage/logs/laravel.log` for scheduler errors

Emails Not Being Sent

1. Check mail configuration in `.env`.
2. Test SMTP: `php artisan tinker --execute 'Mail::raw("test", fn($m) => $m->to("test@example.com")->subject("test"));'`
3. Check `storage/logs/laravel.log` for mail exceptions.
4. Verify the queue worker is running (email notifications are queued — they won't send if the worker is down).
5. Check SMTP credentials and firewall rules (port 587 must be open).

Frontend Not Updating After Deployment

1. Run `npm run build` on the server.
2. Clear browser cache (Ctrl+Shift+R).
3. Check `public/build/manifest.json` exists.
4. If the Vite manifest error appears: rebuild the frontend with `npm run build`.

Database Connection Failures

1. Check `.env` database credentials.
2. Verify the database server is running: `systemctl status mysql`
3. Test connection: `php artisan db:show`
4. Check firewall rules between app server and database server.

Redis Connection Failures

1. Verify Redis is running: `systemctl status redis`
2. Test connection: `redis-cli -h 127.0.0.1 -p 6379 ping`
3. Check `REDIS_*` values in `.env`.
4. The application will fall back to file-based cache if Redis is unavailable, but sessions and queues will not work.

File Uploads Failing

1. Check PHP limits in `php.ini`:

```
upload_max_filesize = 20M
post_max_size = 25M
```

2. Check Nginx client body size:

```
client_max_body_size 25M;
```

3. Verify storage directory is writable by the web server.
4. Check available disk space: `df -h`.

Audit Log "Immutable" Error

The audit log is designed to be immutable. If something is attempting to update or delete an audit log entry, it will throw a `LogImmutableException`. This is intentional behavior — investigate the source of the modification attempt.

Permission Changes Not Taking Effect

After changing roles or permissions:

```
php artisan permission:cache-reset
```

Users may also need to log out and back in.

---

17. Frequently Asked Questions (Admin)

Q: How do I reset a user's password? A: Go to **Administration** → **Users**, open the user, and click **Reset Password**. This sends them a password reset email. Alternatively, use Artisan:

```
php artisan tinker --execute '
$user = \App\Models\User::where("email", "user@example.com")->first();
\Password::sendResetLink(["email" => $user->email]);
echo "Reset link sent.";
'
```

Q: How do I create the first Super Admin? A: See Section 3.4 — Creating the First Super Admin. Use Artisan tinker to create and assign the super_admin role.

Q: Can I have multiple Super Admins? A: Yes. Assign the `super_admin` role to as many users as needed. All will have unrestricted system access.

Q: How do I disable Microsoft SSO login? A: Remove or empty the `AZURE_CLIENT_ID` and `AZURE_CLIENT_SECRET` values in `.env` and restart the application. The SSO button will no longer appear on the login page.

Q: How do I change the system logo? A: Go to **Settings** → **Settings**, upload a new logo image, and save. The logo appears in the header immediately after saving.

Q: How do I add a new college or department? A: Go to **Administration** → **Colleges** (or Departments) and click **New**. Fill in the name and code and save.

Q: How do I prevent users from submitting proposals? A: There are several approaches:

- **Close all proposal calls** — no active call means no submissions.
- **Disable self-registration** — prevents new users from creating accounts.
- **Deactivate specific users** — prevents them from logging in.
- **Maintenance mode** — takes the entire site offline.

Q: The system is running slowly. What should I check? A: In order of likelihood:

1. Check **System Monitor** for CPU, memory, and disk usage.
2. Check for a high number of queued jobs (queue backlog).
3. Clear and rebuild caches: `php artisan cache:clear && php artisan config:cache`.
4. Check `storage/logs/laravel.log` for slow query warnings.
5. Review the performance alerts for any auto-detected anomalies.

Q: How do I update the RTT Platform to a new version? A: Standard update procedure:

1. Enable maintenance mode.
2. `git pull` the new code.
3. `composer install --no-dev`.
4. `php artisan migrate --force`.
5. `npm run build`.
6. `php artisan config:cache && php artisan route:cache && php artisan view:cache`.

7. Restart queue workers: `sudo supervisorctl restart rtt-worker:*`.
8. Disable maintenance mode.

Q: How do I export all proposals to a spreadsheet? A: Go to **Proposals** → **All Proposals**, use filters if needed, and click **Export**. The list downloads as a CSV file that can be opened in Excel or Google Sheets.

Q: How long are audit logs retained? A: Indefinitely by default. To implement a retention policy, create a scheduled command that archives records older than your desired retention period (e.g., 7 years) to an external archive or deletes them. This requires a developer change.

Q: Can I use the system without Redis? A: Not recommended for production. Without Redis:

- Queue jobs will not process (no background notifications or backups)
- Sessions will fall back to file-based storage (won't support max concurrent sessions)
- Cache will fall back to file-based (slower, no shared cache across servers)

For a single-server development setup, install Redis locally (`apt install redis-server`).

Q: How do I configure the system for a multi-server (load-balanced) deployment? A: Ensure:

1. All servers share the same Redis instance (for cache, sessions, queues).
2. All servers share the same file storage (S3 or NFS for `storage/app/`).
3. Database is accessible from all servers.
4. Only one server runs the queue worker (or use a dedicated worker instance).
5. The scheduler (`cron`) runs on only one server.

End of Administrator Manual

Document Information

- System: RTT — Research & Technology Transfer Platform
- Version: 1.0
- Date: June 2026

- For technical support, contact the platform development team.